WEEK 6 SECURITY AUDIT & FINAL DEPLOYMENT REPORT

Name: Aneeqa Kamran
ID: DHC-8189

Week 6 – Advanced Security Audits & Final Deployment Security

Objective

The objective of this lab was to perform advanced security audits, identify vulnerabilities, analyze dependency risks, conduct penetration testing, and prepare the secure application for deployment. Multiple industry-standard security tools were used to evaluate the application's security posture and identify security weaknesses.

The lab also focused on OWASP Top 10 compliance, dependency vulnerability management, and secure deployment preparation using Dockerfile configuration.

Tools Used

The following tools and technologies were used during this lab:

- Windows Command Prompt
- Windows PowerShell
- Node.js
- Express.js
- npm audit
- OWASP ZAP
- Burp Suite Community Edition
- Git & GitHub
- Dockerfile
- Nikto (setup attempted)
- Browser-based localhost testing

Task 1 – Dependency Security Auditing

Step 1.1 – Opening the Week 5 Project

The previously created Week 5 secure Node.js application was used for this lab.

The following command was used to open the project directory:

```
cd C:\sqlmap-dev\week5-lab
```

Step 1.2 – Running npm Audit

The npm audit tool was used to scan Node.js dependencies for known vulnerabilities.

The following command was executed:

```
npm audit
```

Initial Audit Findings

The scan identified multiple vulnerabilities including:

- cookie package vulnerabilities
- csurf dependency issues
- base64-url vulnerability
- uid-safe package risks

The audit reported both low severity and high severity vulnerabilities.

Purpose of npm audit

The purpose of npm audit is to:

- identify insecure dependencies
- detect vulnerable packages
- provide remediation recommendations
- improve application security

Step 1.3 – Fixing Vulnerabilities

The following command was executed to automatically apply available security fixes:

```
npm audit fix
```

After remediation, npm audit was executed again:

```
npm audit
```

Final Audit Results

After remediation:

- several vulnerabilities were fixed successfully
- dependency security improved
- remaining issues were reduced to low severity findings

This demonstrated practical dependency vulnerability management.

Task 2 – Running the Secure Node.js Application

Step 2.1 – Starting the Server

The secure Node.js application created during Week 5 was started using:

```
node app.js
```

The server successfully started on localhost port 3000.

Expected message:

```
Secure Server Running on Port 3000
```

Step 2.2 – Browser Testing

The application was tested using the following URL:

```
http://localhost:3000/form
```

The secure form loaded successfully in the browser.

The form already included:

- SQL Injection protection
- CSRF protection
- input validation
- secure request handling

Task 3 – OWASP ZAP Security Audit

Step 3.1 – Launching OWASP ZAP

OWASP ZAP was opened and configured using the automated scan mode.

Target URL used:

```
http://localhost:3000/form
```

The scan was started using:

- Attack
- Active Scan

Step 3.2 – Vulnerability Scanning

OWASP ZAP scanned the secure application and identified multiple security findings.

Findings Included

- CSP Failure to Define Directive with No Fallback
- Content Security Policy Header Not Set
- Missing Anti-clickjacking Header
- Cookie without SameSite Attribute
- Server Leaks Information via X-Powered-By Header
- X-Content-Type-Options Header Missing
- Session Management Response Identified

Purpose of OWASP ZAP

OWASP ZAP is used to:

- scan web applications
- identify security weaknesses
- detect insecure headers
- analyze sessions and cookies
- test web security configurations

The scan successfully demonstrated security auditing of a web application.

Task 4 – Burp Suite Penetration Testing

Step 4.1 – Opening Burp Suite

Burp Suite Community Edition was launched.

A temporary project was created and the Proxy Intercept feature was enabled.

Step 4.2 – Intercepting Requests

The browser accessed:

```
http://localhost:3000/form
```

HTTP requests were successfully intercepted through Burp Suite.

Captured information included:

- request headers
- cookies
- HTTP GET requests
- localhost traffic
- browser information
- session details

Purpose of Burp Suite

Burp Suite is used for:

- penetration testing
- request interception
- session analysis
- security testing
- vulnerability assessment

The successful interception confirmed that the application traffic could be analyzed securely during penetration testing.

Task 5 – Nikto Configuration Attempt

Nikto setup was attempted in Windows PowerShell.

The following commands were executed:

```
cd C:
git clone https://github.com/sullo/nikto.git
```

Nikto downloaded successfully.

However, Nikto required Perl for execution. Strawberry Perl installation was attempted but installation issues occurred within the Windows environment.

Result

Theoretical Nikto configuration and methodology were documented for the security audit section.

Task 6 – Secure Deployment Preparation

Step 6.1 – Dockerfile Creation

A Dockerfile was created for deployment preparation.

The following Dockerfile configuration was used:

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "app.js"]
```

Purpose of Dockerfile

The Dockerfile was prepared to:

- containerize the application
- support secure deployment
- create a lightweight runtime environment
- improve deployment portability

Although Docker Desktop could not run on the system, deployment preparation was successfully completed.

Task 7 – OWASP Top 10 Compliance Review

The application was reviewed against OWASP Top 10 security best practices.

Security Controls Implemented

- SQL Injection prevention
- CSRF protection
- Input validation
- Secure request handling
- Dependency vulnerability auditing
- Session analysis
- Security header review

These controls improved the overall security posture of the application.

Task 8 – GitHub Repository Update

The Week 6 project updates were uploaded to GitHub.

The following commands were used:

```
git add .
git commit -m "Week 6 security audit and deployment improvements"
git push
```

The repository was successfully updated with:

- Week 6 security improvements
- audit documentation
- deployment preparation
- screenshots and reports

Screenshots Captured

The following screenshots were captured during the lab:

1. npm audit vulnerability scan
2. npm audit fix results
3. OWASP ZAP automated scan
4. Burp Suite request interception
5. Secure localhost application
6. GitHub repository updates

Challenges Faced

Several issues were encountered during the lab:

- Docker Desktop compatibility issues
- Nikto dependency on Perl
- localhost request interception delays
- OWASP ZAP incorrect target URL configuration

All major issues were resolved successfully during testing.

Learning Outcomes

This lab provided practical experience with:

- vulnerability auditing
- penetration testing
- dependency scanning
- security header analysis
- session inspection
- secure deployment preparation
- OWASP Top 10 security practices

The lab also improved understanding of modern web application security testing.

Conclusion

The Week 6 lab successfully demonstrated advanced security auditing and penetration testing techniques. Multiple security tools were used to analyze vulnerabilities, inspect HTTP traffic, review security headers, and improve application security.

The secure Node.js application was tested using OWASP ZAP and Burp Suite, while dependency vulnerabilities were identified and reduced using npm audit.

The lab successfully achieved its objectives related to security auditing, secure deployment preparation, and OWASP compliance analysis.

References

- OWASP ZAP Documentation
- Node.js Documentation
- npm Audit Documentation
- Burp Suite Documentation
- OWASP Top 10
- GitHub Documentation